SECTION A: Conceptual Understanding

1. Why do we need autograd if we already know calculus and can compute derivatives manually?

Answer : In deep learning, neural networks may contain millions of parameters and hundreds of operations. Computing derivatives manually for every parameter becomes extremely difficult, time-consuming, and error-prone. Autograd automatically computes gradients using the chain rule, making training fast and efficient.

2. What problem does autograd solve in deep neural networks with many layers?

Answer : Deep neural networks contain many interconnected layers and parameters. During training, gradients must be computed for every weight and bias. Autograd automatically tracks all operations and computes gradients during backpropagation. This solves the problem of manually calculating derivatives in large and complex networks.

3. What is a computation graph? Explain it using a simple example like: y = (x^2 + 3x) * 2

Answer : A computation graph is a graphical representation of mathematical operations. Each operation is represented step by step so gradients can be computed automatically.
In the given example let a=x^2 , b=3x, c=a+b, y=2c , visually the graph will look like ⇒ x connected to a and b⇒ a and b connected to c ⇒ c connected to multiplied by 2 ⇒ finally ends connected to y as final.

4. In a computation graph, what do nodes represent and what do edges represent?

Answer : Nodes represent variables or mathematical operations. Edges represent the flow of data between operations.

5. Why is the computation graph called a Directed Acyclic Graph (DAG)?

Answer : Directed because data flows in one direction. **Acyclic** because there are no loops or cycles. Information always moves from input toward output during forward propagation.

6. What happens to gradient flow if the graph contains a cycle?

Answer : If the graph contains a cycle:

1. Gradient computation may become infinite or unstable.
2. Backpropagation cannot determine a proper starting and ending point.
3. The chain rule becomes difficult to apply correctly.

This is why autograd system use acyclic graphs.

7. Explain in your own words how the chain rule is applied automatically in autograd.

Answer : Autograd records every mathematical operation during the forward pass. During backward propagation, it starts from the final loss and moves backward through the graph. At each operation, it

computes local derivatives and multiplies them using the chain rule. This automatically gives gradients for all parameters.

SECTION B: Computation Graph and Chain Rule

8. Suppose:
   x = 2
   y = x^2
   z = 3*y
   L = z + 4
   Draw the computation graph and manually compute dL/dx.

Answer : The computational graph : x=2 ⇒ y=x^2 ⇒ z=3y ⇒ L=z+4 . Manually calculation :
dL/dx = d/dx [3x^2 + 4] = 6x
Hence x=2, 6x = 12.

9. If:
   y = sigmoid(w*x + b)
   L = (y - t)^2
   Write the full chain rule expression for dL/dw before simplifying.

Answer : let z=w*x + b
Simplified chain rule ⇒
So, dL/dw = (dL/dy).(dy/dz)(dz/dw)

Further derivations are skipped….

10. Why does Binary Cross Entropy with Sigmoid simplify to (y_pred - y)?

Answer : For Sigmoid + Binary Cross Entropy the derivative of BCE and derivative of Sigmoid mathematically cancel several terms during simplification.
As a result:
dL/dz = y_pred − y equation makes backpropagation simpler and computationally efficient.

11. What would change in the gradient flow if we used ReLU instead of Sigmoid?

Answer : Sigmoid produces gradients between 0 and 1, which can become very small and cause vanishing gradients.
ReLU behaves differently:

I. Positive inputs allow strong gradient flow.
II. Negative inputs produce zero gradient.
III. ReLU reduces vanishing gradient problems compared to Sigmoid.
IV. However, neurons can “die” if outputs remain negative for long periods.